

SalesGenie AI - Team B

API Contract, Tech Stack & Repository Setup — Day 2 Deliverables

1. Tech Stack Finalization

Layer	Technology
Backend / API	FastAPI (Python), Pydantic for schema validation
Database	PostgreSQL, SQLAlchemy ORM
AI Layer	Groq API (LLM inference)
Frontend	Streamlit, Plotly/Altair for charts
DevOps	Docker, GitHub Actions (CI/CD), monitoring via basic health-check endpoints
Testing	Pytest (backend), Postman (API contract verification)

2. Repository & Branching Setup

Repository Structure (as implemented on GitHub)

- SALESGENIE/ (repository root)
 - database/
 - connection.py (DB connection),
 - models.py (SQLAlchemy models)
 - modules/

module1_leads.py — Lead Management & Prospect Database

module2_intelligence.py — Lead Intelligence & Company Analysis

module3_outreach.py — AI Outreach Generation

module4_scoring.py — Lead Scoring & Recommendation Engine

module5_conversation.py — Conversation Intelligence & CRM Integration

module6_dashboard.py — Dashboard & Sales Analytics

- app.py — Streamlit frontend entry point
- main.py — FastAPI backend entry point (mounts routers from modules/)
- seed_data.py — sample data
- run_all.py — convenience script to run backend + frontend together
- requirements.txt
- README.md
- .env / .env.example

Branching Strategy

File / Area	Feature Branch	Primary Owner
database/ (connection.py, models.py)	feature/database-schema	Database / DevOps Engineer
modules/module1_leads.py	feature/module1-leads	Backend Developer
modules/module2_intelligence.py	feature/module2-intelligence	AI/ML Engineer
modules/module3_outreach.py	feature/module3-outreach	AI/ML Engineer
modules/module4_scoring.py	feature/module4-scoring	AI/ML Engineer
modules/module5_conversation.py	feature/module5-conversation	AI/ML Engineer
modules/module6_dashboard.py	feature/module6-dashboard	Frontend Developer + DevOps
app.py (Streamlit frontend)	feature/frontend-app	Frontend Developer
main.py (router mounting)	feature/backend-main	Backend Developer

- main — always deployable; protected, merges only via reviewed PR, only from develop at milestone checkpoints.
- develop — shared integration branch; all feature branches merge here first.
- Each feature branch is created from develop, and merged back into develop via PR once the module's task for the day/milestone is working.
- bugfix/<short-desc> — for fixing a bug found during testing (e.g. bugfix/module4-score-rounding).
- hotfix/<short-desc> — for urgent fixes needed directly on main after a milestone deploy.
- QA/Test Engineer tests each feature branch (or develop after merge) rather than owning a dedicated branch.

3. API Contract — Endpoint List

module1_leads.py — Lead Management & Prospect Database

Method / Endpoint	Description	Request Body (key fields)
GET /leads/stages	Pipeline stage list for dropdowns	—
POST /leads	Create a prospect record	company_name, industry, contact_name, email, phone, deal_value, ...
GET /leads	List / search prospects	query params: q, status, segment
GET /leads/{lead_id}	Get one lead's full profile	—
PUT /leads/{lead_id}	Update a lead record	any editable lead fields incl. deal_value
DELETE /leads/{lead_id}	Delete a lead record	—
POST /leads/{lead_id}/interactions	Log a manual engagement note	interaction_type, summary, action_items
GET /leads/{lead_id}/interactions	Get engagement history for a lead	—

module2_intelligence.py — Lead Intelligence & Company Analysis

Method / Endpoint	Description	Request Body (key fields)
GET /intelligence/health	LLM configuration health check	—
POST /intelligence/generate/{lead_id}	AI qualification score, business needs, opportunities, industry fit, reasoning	— (reads stored lead data)
GET /intelligence/{lead_id}	Latest saved insight for a lead	—
GET /intelligence/{lead_id}/history	All past insights for a lead	—

module3_outreach.py — AI Outreach Generation

Method / Endpoint	Description	Request Body (key fields)
GET /outreach/health	LLM configuration health check	—
POST /outreach/generate	Generate a personalized cold email / follow-up	lead_id, email_type, tone
POST /outreach/save	Save a (possibly edited) email as draft or sent	lead_id, subject, body, status
GET /outreach/history/{lead_id}	Past campaigns for a lead	—
PATCH /outreach/mark-sent/{campaign_id}	Mark a saved draft as sent	—

module4_scoring.py — Lead Scoring & Recommendation Engine

Method / Endpoint	Description	Request Body (key fields)
GET /scoring/status	Module health/status check	—
POST /scoring/generate/{lead_id}	Numeric score, classification (Platinum/Gold/Silver/Bronze), AI recommendations	—
GET /scoring/{lead_id}	Latest saved score for a lead	—
POST /scoring/simulate	What-If simulator — recompute score against hypothetical inputs	lead_id, simulated feature overrides
GET /scoring/ranking/list	All scored leads ranked by score, for Priority Rankings tab	—

module5_conversation.py — Conversation Intelligence & CRM Integration

Method / Endpoint	Description	Request Body (key fields)
GET /conversation/status	Module health/status check	—
GET /conversation/health	LLM configuration health check	—
GET /conversation/crm-platforms	List of supported CRM platforms (Salesforce, HubSpot)	—
POST /conversation/sync/{lead_id}	Sync a lead to a CRM platform (simulated — no live credentials configured)	crm_platform
GET /conversation/sync/logs/{lead_id}	Sync history for one lead	—
GET /conversation/sync/logs	Recent sync activity across all leads	—
POST /conversation/summarize	AI summary + action items from a call/meeting transcript	lead_id, transcript, interaction_type
GET /conversation/summaries/{lead_id}	Past AI-generated summaries for a lead	—

module6_dashboard.py — Dashboard & Sales Analytics

Method / Endpoint	Description	Request Body (key fields)
GET /dashboard/status	Module health/status check	—
GET /dashboard/overview	KPIs: total leads, conversion rate, pipeline value, avg scores, campaign/interaction counts	—
GET /dashboard/pipeline	Kanban-style lead breakdown by pipeline stage	—
GET /dashboard/followups	Rule-based automated follow-up feed (no LLM call — reads Module 4 + Module 1 data)	—